Trabalho 1 — Sistemas Operacionais – 2026/1
O Duelo de Contextos: Processos vs. Threads

Arthur Pinto Ferreira[1], **Matheus Corbellini Steindorff**[1],
Rafaela do Nascimento Mello[1], **Vinicius Moço Quintian**[1]

[1]Disciplina de Sistemas Operacionais – 2026/1
Pontifícia Universidade Católica do Rio Grande do Sul (PUCRS)
Porto Alegre – RS – Brasil

Resumo. *Este trabalho compara o overhead de criação, o custo de comunicação e a consistência de dados entre processos (POSIX fork) e threads (POSIX pthreads) em ambiente Linux. O experimento incrementa um contador global até 1.000.000.000, distribuindo o trabalho entre N unidades de execução (N = 2, 4 e 8), em quatro cenários: threads sem mutex (T1), threads com mutex (T2), processos sem semáforo (P1) e processos com semáforo (P2).*

Abstract. *This paper compares the creation overhead, communication cost, and data consistency between processes (POSIX fork) and threads (POSIX pthreads) on Linux. The experiment increments a global counter to 1,000,000,000, distributing the work among N execution units (N = 2, 4, and 8) across four scenarios: threads without mutex (T1), threads with mutex (T2), processes without semaphore (P1), and processes with semaphore (P2).*

1. Introdução

Este trabalho compara o overhead de criação, o custo de comunicação e a consistência de dados entre processos (POSIX fork) e threads (POSIX pthreads) em ambiente Linux.

O experimento consiste em incrementar um contador global até 1.000.000.000 (um bilhão), distribuindo o trabalho entre N unidades de execução (N = 2, 4 e 8), em quatro cenários descritos na Tabela 1.

Table 1. Cenários do experimento

Código	Descrição
T1	Threads sem mutex (sem sincronização)
T2	Threads com `pthread_mutex`
P1	Processos via `fork` + memória compartilhada (`shmget`) sem semáforo
P2	Processos via `fork` + memória compartilhada com `sem_open`

Os comandos utilizados para compilar e executar os experimentos são:

```
make                # Compilar
lscpu               # Assinatura do Hardware
./threads <N> 1     # Executar threads sem mutex (T1)
./threads <N> 2     # Executar threads com mutex (T2)
./processos <N> 1 # Executar processos sem semaforo (P1)
```

```
./processos <N> 2 # Executar processos com semaforo (P2)
make clean        # Limpar binarios
```

2. Desenvolvimento

2.1. Assinatura do Hardware

O processador utilizado é um Intel Core i3-N305 com 8 núcleos físicos e 1 thread por núcleo (sem Hyper-Threading). A saída completa do `lscpu` é apresentada a seguir:

```
Architecture:       x86_64
  CPU op-mode(s):   32-bit, 64-bit
  Address sizes:    39 bits physical, 48 bits virtual
  Byte Order:       Little Endian
CPU(s):             8
  On-line CPU(s) list: 0-7
Vendor ID:          GenuineIntel
  Model name:       Intel(R) Core(TM) i3-N305
    CPU family:     6
    Model:          190
    Thread(s) per core: 1
    Core(s) per socket: 8
    Socket(s):      1
    Stepping:       0
    BogoMIPS:       3609.61
Virtualization features:
  Virtualization:   VT-x
  Hypervisor vendor: Microsoft
  Virtualization type: full
Caches (sum of all):
  L1d:              256 KiB (8 instances)
  L1i:              512 KiB (8 instances)
  L2:               16 MiB (8 instances)
  L3:               6 MiB (1 instance)
```

Isso significa que os experimentos com N = 8 podem saturar todos os núcleos disponíveis, o que impacta diretamente no comportamento de concorrência e na frequência de colisões observadas nos experimentos sem sincronização.

2.2. Resultados

2.2.1. Tabela Completa (Tempo + Valor Final)

A Tabela 2 reúne todos os resultados obtidos, apresentando simultaneamente o tempo de execução e o valor final atingido pelo contador para cada combinação de experimento e número de trabalhadores N.

2.2.2. Tabela de Valores Finais

A Tabela 3 isola o valor final do contador para cada cenário, facilitando a comparação direta da integridade dos dados entre os experimentos com e sem sincronização.

Table 2. Resultados completos — tempo e valor final do contador

Experimento	N	Tempo (s)	Valor Final
T1 — threads sem mutex	2	3,3047	654.290.134
T1 — threads sem mutex	4	7,5468	684.678.724
T1 — threads sem mutex	8	17,6189	447.515.488
T2 — threads com mutex	2	99,6028	1.000.000.000
T2 — threads com mutex	4	93,5196	1.000.000.000
T2 — threads com mutex	8	90,7124	1.000.000.000
P1 — processos sem semáforo	2	2,5497	533.402.362
P1 — processos sem semáforo	4	2,8438	319.062.540
P1 — processos sem semáforo	8	3,1545	203.808.717
P2 — processos com semáforo	2	182,4934	1.000.000.000
P2 — processos com semáforo	4	200,6238	1.000.000.000
P2 — processos com semáforo	8	303,9470	1.000.000.000

Table 3. Valor final do contador por experimento e número de trabalhadores

Experimento	N = 2	N = 4	N = 8
T1 — threads sem mutex	654.290.134	684.678.724	447.515.488
T2 — threads com mutex	1.000.000.000	1.000.000.000	1.000.000.000
P1 — processos sem semáforo	533.402.362	319.062.540	203.808.717
P2 — processos com semáforo	1.000.000.000	1.000.000.000	1.000.000.000

2.2.3. Valores com Corrupção (abaixo de 1 bilhão)

A Tabela 4 destaca exclusivamente os experimentos que não atingiram o valor esperado de 1.000.000.000, evidenciando a magnitude da corrupção de dados causada pela ausência de sincronização.

Table 4. Experimentos com corrupção de dados

Experimento	N = 2	N = 4	N = 8
T1 — threads sem mutex	654.290.134	684.678.724	447.515.488
P1 — processos sem semáforo	533.402.362	319.062.540	203.808.717

2.2.4. Tabela de Tempos

A Tabela 5 apresenta somente os tempos de execução, permitindo comparar diretamente o desempenho temporal entre os quatro cenários e os diferentes valores de N.

2.3. Análise de Corrupção

Nos experimentos T1 (threads sem mutex) e P1 (processos sem semáforo), o contador não atingiu 1.000.000.000, conforme apresentado na Tabela 4.

Table 5. Tempo de execução por experimento e número de trabalhadores

Experimento	N = 2	N = 4	N = 8
T1 — threads sem mutex	3,3047 s	7,5468 s	17,6189 s
T2 — threads com mutex	99,6028 s	93,5196 s	90,7124 s
P1 — processos sem semáforo	2,5497 s	2,8438 s	3,1545 s
P2 — processos com semáforo	182,4934 s	200,6238 s	303,9470 s

2.3.1. Por que o contador não chegou a 1 bilhão?

A operação `contador++` não é atômica. Em nível de máquina ela se decompõe em três etapas: leitura do valor atual, incremento em registrador e escrita do resultado. Quando múltiplas unidades de execução realizam essas etapas simultaneamente sem sincronização, ocorre uma condição de corrida (*race condition*):

1. Unidade A lê `contador = 1000`.
2. Unidade B lê `contador = 1000` (antes de A escrever).
3. A escreve `contador = 1001`.
4. B escreve `contador = 1001` — o incremento de A foi sobrescrito e perdido.

   Esse fenômeno é chamado de *lost update* (atualização perdida).

2.3.2. Correlação com o hardware

O i3-N305 possui 8 núcleos independentes sem Hyper-Threading, o que significa que com N = 8 todos os núcleos executam genuinamente em paralelo, sem compartilhamento de pipeline. Isso maximiza as janelas de colisão entre as leituras e escritas concorrentes, explicando por que P1 com N = 8 (203.808.717) teve o pior índice de corrupção — quase 80% das incrementações foram perdidas. Cada núcleo possui seu próprio cache L1/L2 privado (256 KiB e 16 MiB por instância), o que agrava a inconsistência: cada núcleo pode ter uma cópia diferente do contador em seu cache antes de propagá-la para o L3 compartilhado.

2.4. Gráfico de Escalabilidade

2.4.1. Gráfico 1 — Tempo de Execução vs. Número de Trabalhadores (N)

O gráfico de linhas mostra o tempo de execução (em segundos) para cada cenário conforme N varia entre 2, 4 e 8 trabalhadores. Os cenários com sincronização (T2 e P2) dominam o eixo Y: P2 sobe de $\sim$182 s (N=2) até $\sim$304 s (N=8), pois cada incremento exige uma chamada de sistema de semáforo, que escala pessimamente com mais processos concorrendo pelo mesmo recurso. T2 apresenta comportamento levemente decrescente ($\sim$99 s $\rightarrow$ $\sim$91 s), pois o mutex de thread tem overhead fixo mais baixo e a contenção se distribui melhor entre os núcleos. Os cenários sem sincronização (T1 e P1) ficam próximos de zero na escala do gráfico: P1 mal ultrapassa 3 s mesmo em N=8, e T1 chega a $\sim$18 s em N=8 — tempos baixos que resultam diretamente da corrupção de dados (menos incrementos efetivos sendo contabilizados).

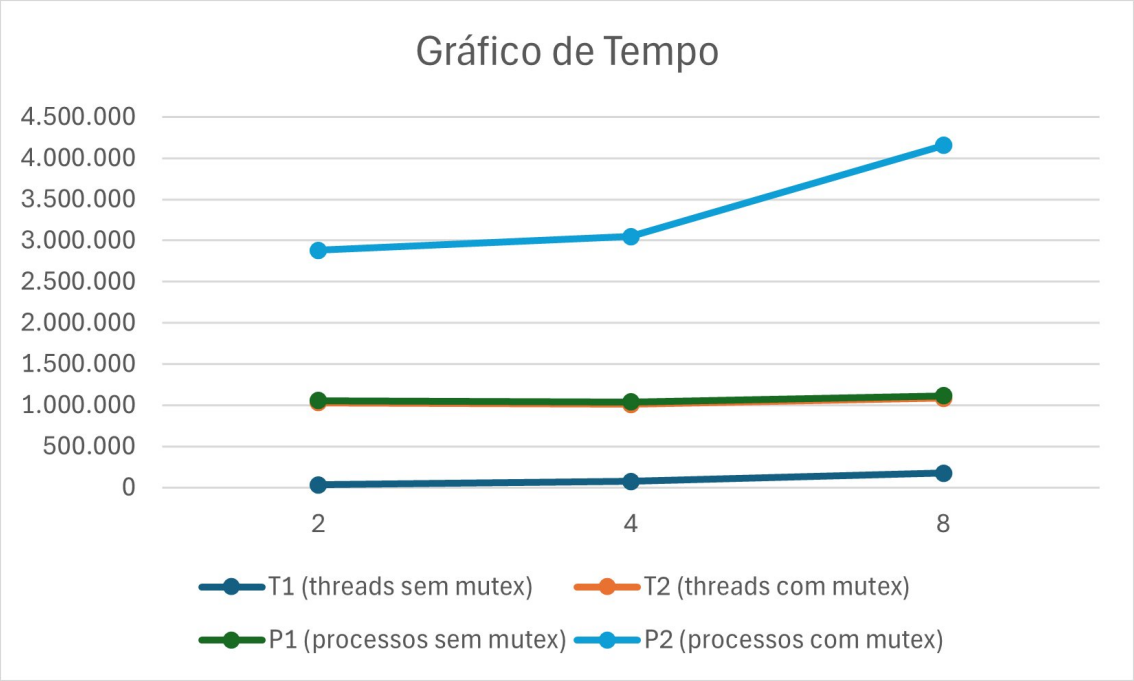

Figure 1. Tempo de execução (s) por cenário vs. número de trabalhadores (N)

2.4.2. Gráfico 2 — Valor Final do Contador vs. Número de Trabalhadores (N)

O gráfico de barras compara o valor final alcançado pelo contador em cada cenário e valor de N. T2 e P2 (com sincronização) atingem consistentemente 1.000.000.000 em todos os valores de N, confirmando que os mecanismos de exclusão mútua garantem a integridade dos dados. T1 e P1 (sem sincronização) apresentam valores muito abaixo de 1 bilhão, com degradação crescente conforme N aumenta: P1 cai de ∼533 milhões (N=2) para apenas ∼204 milhões (N=8), evidenciando que mais núcleos paralelos amplificam as colisões de leitura/escrita na memória compartilhada. T1 oscila entre ∼447 e ∼685 milhões, com comportamento menos previsível devido à forma como o escalonador distribui as threads entre os núcleos.

3. Conclusão

3.1. Overhead de Criação

Criar um processo via `fork` tem custo maior que criar uma thread: o sistema operacional precisa duplicar o espaço de endereçamento do processo pai (*copy-on-write*), alocar uma entrada na tabela de processos e configurar canais de IPC. Threads compartilham o mesmo espaço de endereçamento do processo criador, tornando sua criação significativamente mais leve.

Nos experimentos sem sincronização, P1 foi mais rápido que T1 em todos os cenários, o que pode parecer contraditório. Porém, isso se deve à menor corrupção de cache entre processos no início da execução — cada processo tem espaço próprio — e não indica menor overhead de criação.

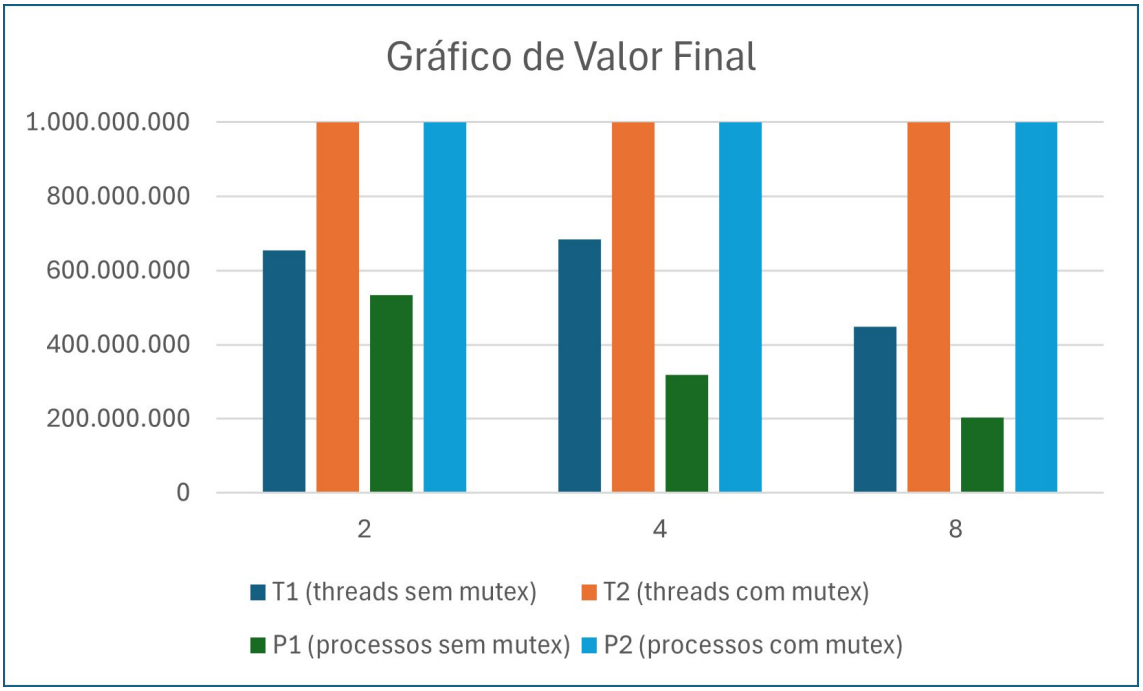

Figure 2. Valor final do contador por cenário vs. número de trabalhadores (N)

3.2. Comunicação

Threads compartilham memória automaticamente via variáveis globais. A comunicação é implícita e de baixo custo, mas exige sincronização explícita para evitar *race conditions*.

Processos precisam de mecanismos explícitos de IPC como memória compartilhada (`shmget`/`shmat`). Há custo adicional de configuração e de chamadas de sistema, além da necessidade de semáforos para proteger o acesso. Threads são claramente mais eficientes na comunicação para dados compartilhados.

3.3. Consistência de Dados

Table 6. Consistência dos resultados com N = 8

Cenário	Resultado com N = 8	Consistente?
T1	447.515.488	Não
T2	1.000.000.000	Sim
P1	203.808.717	Não
P2	1.000.000.000	Sim

Sem mecanismos de sincronização (mutex ou semáforo), nenhum modelo garante consistência. Com sincronização, ambos atingem o resultado correto, porém com alto custo de tempo, especialmente P2 (semáforos POSIX nomeados têm overhead de chamada de sistema maior que mutexes de threads).

3.4. Comparativo Final

Processos têm maior overhead de criação e de comunicação. Threads são mais eficientes para tarefas que compartilham dados, desde que a sincronização seja aplicada corretamente.

Table 7. Comparativo final entre threads e processos

Aspecto	Threads (pthreads)	Processos (fork)
Overhead de criação	Baixo	Alto
Comunicação	Eficiente (memória compartilhada nativa)	Custosa (IPC explícita via `shmget`)
Consistência sem sinc.	Corrompida	Corrompida (mais severa com mais núcleos)
Consistência com sinc.	Sim (mutex) — $\sim$95 s	Sim (semáforo) — 180–304 s